

credit_card_fraud.ipynb

Jupyter Notebook — End-to-End Fraud Detection Walkthrough

■ What This Notebook Does

This notebook is a fully self-contained, interactive fraud detection pipeline. It downloads its own data, preprocesses it, trains a Random Forest with SMOTE balancing, evaluates it, and shows a Confusion Matrix — all in 3 cells you can run top-to-bottom. It is the interactive, exploratory version of model.py.

■ Cell 1 — Markdown Title

```
# Credit Card Fraud Prediction using Random Forest & SMOTE
```

This is a Markdown cell (not code). It renders as a formatted heading in the notebook, describing what the notebook does. It has no effect on execution.

■ Cell 2 — Imports & Data Download

```
import urllib.request
import os
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (classification_report, confusion_matrix,
                             precision_recall_curve, auc)

import matplotlib.pyplot as plt
import seaborn as sns
from imblearn.over_sampling import SMOTE

DATA_FILE = 'creditcard.csv'
DATA_URL = 'https://storage.googleapis.com/download.tensorflow.org/data/creditcard.csv'

if not os.path.exists(DATA_FILE):
    print(f'Downloading data from {DATA_URL}...')
    urllib.request.urlretrieve(DATA_URL, DATA_FILE)
else:
    print(f'{DATA_FILE} already exists. Skipping download.')

df = pd.read_csv(DATA_FILE)
df.head()
```

How this cell works:

urllib.request

Python's built-in HTTP module. Used instead of requests/wget so no extra library is needed.
urllib.retrieve(url, filename) downloads the file at the URL and saves it with the given filename — one line, no loops.

DATA_URL points to TensorFlow's public storage

The creditcard.csv is hosted on Google Cloud Storage by TensorFlow. It is publicly accessible (~150 MB). This is the standard Kaggle Credit Card Fraud dataset with 284,807 transactions, 492 of which are fraud.

if not os.path.exists(DATA_FILE):

Checks if the CSV is already on disk. If it is, skips the download (saves ~150 MB and time). If not, downloads it. This makes the notebook re-runnable without re-downloading each time.

urllib.request.urlretrieve(DATA_URL, DATA_FILE)

Makes a GET request to the URL and writes the response body directly to creditcard.csv. This is a blocking call — the cell waits until the entire file is downloaded before continuing.

df = pd.read_csv(DATA_FILE)

Loads all 284,807 rows into a pandas DataFrame. Columns: Time (seconds since first transaction), V1–V28 (PCA-anonymised features), Amount (transaction value in dollars), Class (0=legit, 1=fraud).

df.head()

The last expression in a Jupyter cell is automatically displayed as a formatted HTML table below the cell. Shows the first 5 rows as a quick sanity check that the data loaded correctly.

■ Cell 3 — Preprocess, Train & Evaluate

```
# Preprocess and Train
df = df.dropna()
X = df.drop('Class', axis=1)
y = df['Class']

print(f'Original dataset shape: {y.value_counts().to_dict()}')

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

print('Applying SMOTE...')
smote = SMOTE(random_state=42)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)
print(f'Resampled: {pd.Series(y_train_res).value_counts().to_dict()}')

print('Training Random Forest...')
model = RandomForestClassifier(n_estimators=50, random_state=42, n_jobs=-1)
model.fit(X_train_res, y_train_res)

predictions = model.predict(X_test)
probs = model.predict_proba(X_test)[: , 1]

print('Classification Report:')
print(classification_report(y_test, predictions))
```

```

cm = confusion_matrix(y_test, predictions)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.show()

```

How this cell works — step by step:

df.dropna()

Removes any rows with missing values. The creditcard dataset has none, but this is a safety guard that prevents cryptic errors during training if the data ever changes.

X = df.drop('Class', axis=1) / y = df['Class']

X is the feature matrix — every column except the label. Shape: (284807, 30). y is the label vector — 0 or 1. y.value_counts() will show something like {0: 284315, 1: 492} — showing the extreme imbalance.

train_test_split(..., test_size=0.2, stratify=y)

Randomly splits: 80% → training (227,845 rows), 20% → testing (56,962 rows). stratify=y is critical: it ensures both train and test sets contain the same ~0.17% fraud ratio. Without this, the random split might accidentally put all fraud in training, leaving nothing to test on.

SMOTE(random_state=42).fit_resample(X_train, y_train)

The training set has ~227,451 legit vs ~394 fraud rows. A model trained here would just predict 'legit' always and score 99.8% accuracy — but catch 0 fraud. SMOTE fixes this: it generates ~227,000 synthetic fraud samples by interpolating between existing real fraud points. The result is a balanced 50/50 training set. random_state=42 makes the synthetic generation reproducible.

RandomForestClassifier(n_estimators=50, n_jobs=-1).fit(...)

Trains 50 decision trees simultaneously using all CPU cores (n_jobs=-1). Each tree: (1) samples rows with replacement (bootstrap), (2) at each split, considers only a random subset of features, (3) grows until leaves are pure or max depth reached. Prediction = majority vote of all 50 trees.

model.predict(X_test)

Each of the 50 trees votes for each test row. Majority wins. Returns an array of 56,962 predictions (0s and 1s).

model.predict_proba(X_test)[:, 1]

Returns the fraction of trees that voted 'fraud' for each row (a probability between 0 and 1).[:, 1] extracts the fraud probability column. Used by the PR curve.

classification_report(y_test, predictions)

Prints a table with precision, recall and F1 for each class. For fraud (class 1): Recall = TP/(TP+FN) — what % of actual fraud did we catch. Precision = TP/(TP+FP) — of transactions flagged as fraud, what % were real. F1 = harmonic mean of the two. A good model aims for high recall on class 1.

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

Draws the 2x2 confusion matrix as a colour-coded heatmap. annot=True writes the actual numbers inside the boxes. fmt='d' formats them as integers. cmap='Blues' uses a blue gradient — darker = higher count. plt.show() renders it inline in the notebook output area.

■ Full Data Flow Summary

1 Load libraries & define paths

All imports are done once. DATA_FILE and DATA_URL are set as constants.

2 Auto-download data (if needed)

urllib checks for the CSV on disk. If missing, downloads ~150 MB from TensorFlow storage.

3 Load into DataFrame

pd.read_csv loads 284,807 rows × 31 columns. df.head() displays the first 5 rows.

4 Clean & split features/labels

dropna() removes nulls. X gets 30 feature columns, y gets the Class column (0/1).

5 Train/test split (80/20, stratified)

Training set: ~227,845 rows. Test set: ~56,962 rows. Fraud ratio preserved in both.

6 SMOTE oversampling

Generates synthetic fraud samples in training set → balanced 50/50 split (~454,000 rows).

7 Train Random Forest (50 trees)

model.fit() trains on the balanced SMOTE dataset using all CPU cores.

8 Predict on test set

Hard predictions (0/1) and fraud probabilities generated for all 56,962 test rows.

9 Print classification report

Shows precision, recall, F1 for legit and fraud classes separately.

10 Plot Confusion Matrix

2×2 heatmap rendered inline in the notebook showing TP, FP, TN, FN counts.